MODULE 3

Timers/Counters & Serial port programming: Basics of Timers & Counters, Data types & Time delay in the 8051 using C, Programming 8051 Timers, Mode 1 & Mode 2 Programming, Counter Programming (Assembly Language only). (Text book 2- 3.4, Text book 1- 7.1, 9.1,9.2) Basics of Serial Communication, 8051 Connection to RS232, Programming the 8051 to transfer data serially & to receive data serially using C.(Text book 2- 3.5, Text book 1- 10.1,10.2,10.3 except assembly language programs, 10.5)

8051 Timers

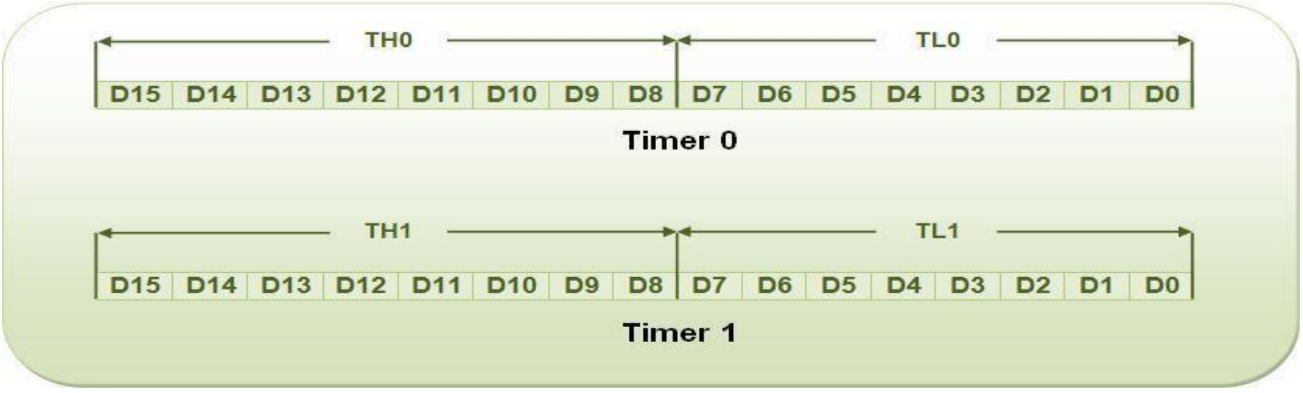
The 8051 has two timers/counters, they can be used either as

i) Timers to generate a time delay or as
ii) Event counters to count events happening outside the microcontroller ‰

Both Timer 0 and Timer 1 are 16 bits wide and Since 8051 has an 8-bit architecture, each 16-bits timer is accessed as two separate registers of low byte and high byte. The low byte register is called TL0/TL1 and the high byte register is called TH0/TH1 and they are accessed like any other register

MOV TL0,#4FH

MOV R5,TH0



The two timers in 8051 share two SFRs (TMOD and TCON) which control the timers, and each timer also has two SFRs dedicated solely to itself (TH0/TL0 and TH1/TL1).

The following are timer related SFRs in 8051.

SFR Name	Description	SFR Address
TH0	*Timer 0 High Byte*	*8Ch*
TL0	*Timer 0 Low Byte*	*8Ah*

TH1	*Timer 1 High Byte*	*8Dh*
TL1	*Timer 1 Low Byte*	*8Bh*
TCON	*Timer Control*	*88h*
TMOD	*Timer Mode*	*89h*

TMOD : Timer/Counter Mode Control Register (Not Bit Addressable)

GATE	C/$\overline{T}$	M1	M0	GATE	C/$\overline{T}$	M1	M0
TIMER 1				TIMER 0			

GATE — When TRx (in TCON) is set and GATE = 1, TIMER/COUNTERx will run only while INTx pin is high (hardware control). When GATE = 0, TIMER/COUNTERx will run only while TRx = 1 (software control).

C/$\overline{T}$ — Timer or Counter selector. Cleared for Timer operation (input from internal system clock). Set for Counter operation (input from Tx input pin).

M1 — Mode selector bit (NOTE 1).

M0 — Mode selector bit (NOTE 1).

Note 1 :

M1	M0	OPERATING MODE	
0	0	0	13–bit Timer
0	1	1	16–bit Timer/Counter
1	0	2	8–bit Auto–Reload Timer/Counter
1	1	3	(Timer 0) TL0 is an 8–bit Timer/Counter controlled by the standard Timer 0 control bits, TH0 is an 8–bit Timer and is controlled by Timer 1 control bits.
1	1	3	(Timer 1) Timer/Counter 1 stopped.

TCON : Timer/Counter Control Register (Bit Addressable)

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

TF1 TCON.7 — Timer 1 overflow flag. Set by hardware when the Timer/Counter 1 overflows. Cleared by hardware as processor vectors to the interrupt service routine.

TR1 TCON.6 — Timer 1 run control bit. Set/cleared by software to turn Timer/Counter ON/OFF.

TF0 TCON.5 — Timer 0 overflow flag. Set by hardware when the Timer/Counter 0 overflows. Cleared by hardware as processor vectors to the service routine.

TR0 TCON.4 — Timer 0 run control bit. Set/cleared by software to turn Timer/Counter 0 ON/OFF.

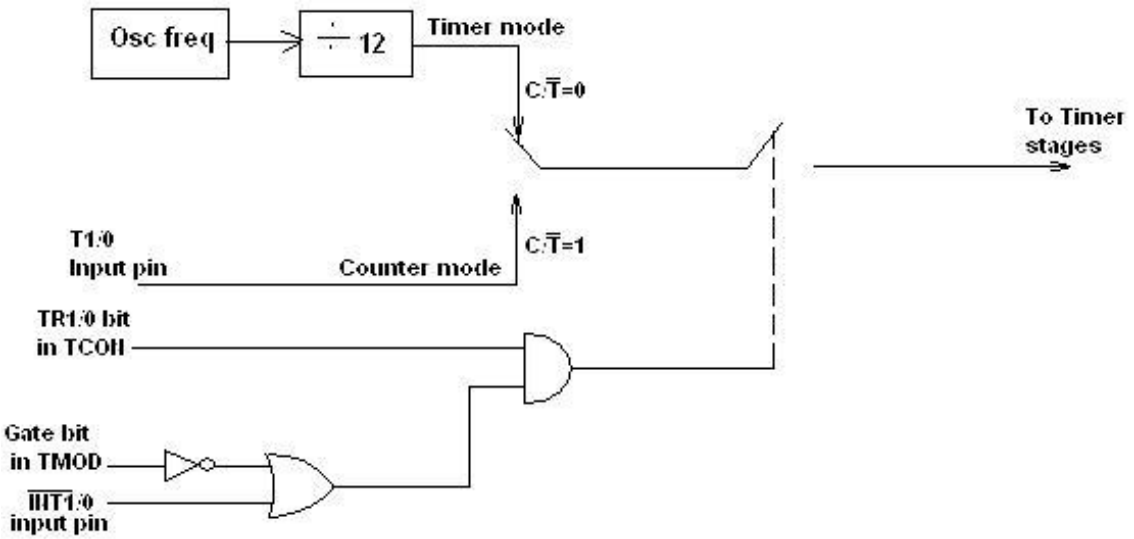
IE1 TCON.3 — External Interrupt 1 edge flag. Set by hardware when External interrupt edge is detected. Cleared by hardware when interrupt is processed.

IT1 TCON.2 — Interrupt 1 type control bit. Set/cleared by software to specify falling edge/flow level triggered External Interrupt.

IE0 TCON.1 — External Interrupt 0 edge flag. Set by hardware when External Interrupt edge detected. Cleared by hardware when interrupt is processed.

IT0 TCON.0 — Interrupt 0 type control bit. Set/cleared by software to specify falling edge/low level triggered External Interrupt.

Timer/ Counter Control Logic.



C/T used to select the clock source to the timer.

TIMER MODES

Timers can operate in four different modes. They are as follows

Timer Mode-0: In this mode, the timer is used as a 13-bit UP counter as follows.

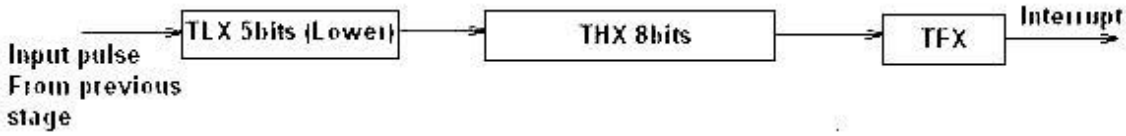


Fig. Operation of Timer on Mode-0

The lower 5 bits of TLX and 8 bits of THX are used for the 13 bit count. Upper 3 bits of TLX are ignored. When the counter rolls over from all 0's to all 1's, TFX flag is set and an interrupt is generated. The input pulse is obtained from the previous stage. If TRx bit is 1 and Gate bit is 0, the counter continues counting up. If TRx bit is 1 and Gate bit is 1, then the operation of the counter is controlled by external input. This mode is useful to measure the width of a given pulse fed to input.

Timer Mode-1: This mode is similar to mode-0 except for the fact that the Timer operates in 16-bitmode. If TRx bit is 1 and Gate bit is 0, the counter continues counting up for each clock cycle (i.e) 12/Xtal freq. If TRx bit is 1 and Gate bit is 1, then the operation of the counter is controlled by external input.

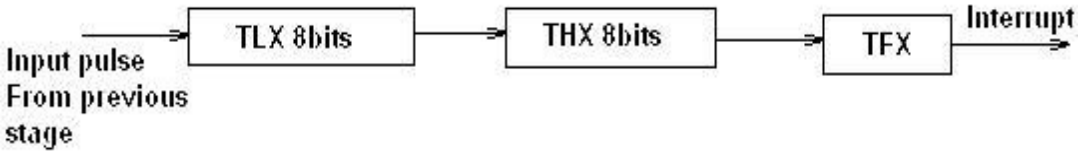


fig: Operation of Timer in Mode 1

When TRx bit is set 1 and gate bit 0, then the timer starts counting from its initial count loaded in THx and TLx to its final value FFFFh. When it rolls over from FFFFH to 0000, it sets a flag bit called TFx (timer flag) to high–Each timer has its own timer flag: TF0 for timer 0, and TF1 for timer 1. –
This timer overflow flag (TFx) can be monitored, When this timer flag is raised, one option would be to stop the timer with the instructions CLR TR0or CLR TR1, for timer 0 and timer 1, respectively.After the timer reaches its limit and rolls over, in order to repeat the process TH and TL must be reloaded with the original value, and TF must be cleared to 0

Timer Mode-2: (Auto-Reload Mode): This is a 8 bit counter/timer operation. Counting isperformed in TLX while THX stores a constant value. In this mode when the timer overflows i.e. TLX becomes FFH, it is reloaded again with the value stored in THX. For example if we load THX with 50H then the timer in mode 2 will count from 50H to FFH. After the count value changes from FFh to 00h, TFx flag is raised and the timer is again reloaded with 50H. This mode is useful in applications like serial communication &fixed time sampling.

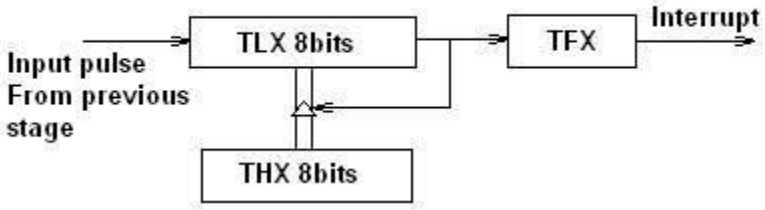


Fig: Operation of Timer in Mode 2

Timer Mode-3: Timer 1 in mode-3 simply holds its count. The effect is same as setting TR1=0.

Timer0 in mode-3 establishes TL0 and TH0 as two separate counters.

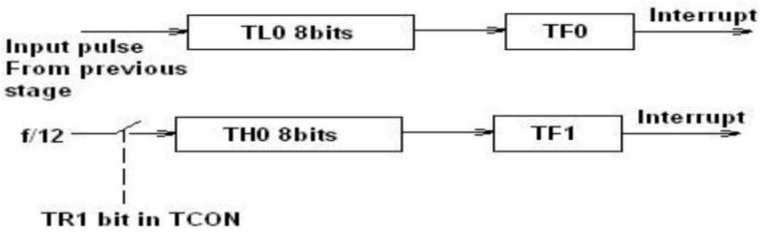


Fig: Operation of Timer in Mode 3

Control bits TR1 and TF1 are used by Timer-0 (higher 8 bits) (TH0) in Mode-3 while TR0 and TF0 are available to Timer-0 lower 8 bits(TL0).

PROGRAMMING 8051 TIMERS IN ASSEMBLY

In order to program 8051 timers, it is important to know the calculation of initial count value to be stored in the timer register. The calculations are as follows.

In any mode, Timer Clock period = 1/Timer Clock Frequency.

= (1/XTAL Clock Frequency)*12 if XTAL frequency is 12MHz, then the clock period is (1/12MHz)*12=1μs

a. Mode 1 (16 bit timer/counter)

- Initial count Value to be loaded to timer(in decimal) = 65536 – (Delay Required/Timer clock period)
- Convert the answer into hexadecimal number and load the higher byte of the initial count onto THx and lower byte of the count to TLx register.

b. Mode 0 (13 bit timer/counter)

- Initial count Value to be loaded in decimal to timer = 8192 – (Delay Required/Timer clock period)
- Convert the answer into hexadecimal and load onto THx and TLx register.

c. Mode 2 (8 bit auto reload)

- Value to be loaded in decimal = 256 – (Delay Required/Timer clock period)
- Convert the answer into hexadecimal number and load onto THx register. Upon starting thetimer this value from THx will be reloaded to TLx register.

Steps to program timer in mode 1 for generating a time delay

1. Load the TMOD value register indicating which timer (timer 0 or timer 1) is to be used and which timer mode (0 or 1) is selected

2. Load registers TLx and THx with initial count value

3. Start the timer by setting the TRx bit of TCON Register using **SETB TRx**(i.e) TRx=1

4. Keep monitoring the timer overflow flag (TFx) with the **JNB TFx,target**instruction to see if it is raised and Get out of the loop when TFx becomes high

5. Stop the timer by clearing the TRx bit using CLR TRx(i.e) TRx=0

6. Clear the TFx flag for the next round

7.Go back to Step 2 to load THx and TLx again if delay generation has to be repeated.

1. ***Write a program to generate a pulse on pin P1.5 for a pulse width of 200μs using timer 1. Assume the crystal oscillator frequency to be 12 MHz.***

Required Pulse width= 250 μs.

The value n for 250 μs is = 250 μs /1 μs = 250

Initial Count to be loaded to timer1 = 65536 - 250 = 65286

Hexadecimal value for the initial count = (65286)d = FF06H.

Hexadecimal value to be Loaded in TL1 = 06H and

Hexadecimal value to be Loaded in TH1 = 0FFH.

Since Timer1 to be used for delay generation TMOD value = 10h

Assembly Program

```
ORG 00H
MOV TMOD, #10H
MOV TH1, #0FFH
MOV TL1, #06H
SETB P1.5
SETB TR1
L1: JNB TF1, L1
CLR P1.5
CLR TR1
CLR TF1
STOP: SJMP STOP
END
```

C Program

```
#include <reg51.h>

int main()

{

TMOD=0X10;

TH1=0XFF;

TL1=0X06;

P1.5=1;

TR1=1;

while(TF1==0);

P1.5=0;

TR1=0;

TF1=0;

}
```

In the above program notice the following step.

1. TMOD is loaded with 10 as Timer1 to be used
2. FF06H is loaded into TH1&TL1.
3. Set the pin P1.5
4. Timer 1 is started by the SETB TR1instruction.
5. Timer 1 counts up with the passing of each clock, which is provided by the crystal oscillator. As the timer counts up, it goes through the states of FF06, FF07, FF08, FF09, FF0A, FF0B, and so on until it reaches FFFFH. In one more clock, the Timer1 rolls its countto 0, raising the timer overflow flag (TF1=1). At that point, the JNB instruction fails through.
6. Clear the pin P1.5.
7. Stop the timer
8. Clear the timer overflow flag.

Go back to Step 2 to load THx and TLx again if delay generation has to be repeated.

2. ***Write a program to generate a pulse on pin P1.5 for a pulse width of 50ms using timer 0. Assume the crystal oscillator frequency to be 11.0592MHz.***

Timer Clock period = 12*(1/11.0592MHz) = 1.085μs

Pulse width= 50 ms.

The value n (initial count) for 50ms is: 50ms /1.085 μs = 46083;

Initial count = 65536 - 46083 = 19453

$(19453)_{10} = (4BFD)_{hex}$. Therefore TL0 = 0FDH and TH0 = 4BH.

Since Timer0 to be used for delay generation TMOD value = 01h

```
ORG 00H
MOV TMOD, #01H
MOV TH0, #4BH
MOV TL0, #0FDH
SETB P1.5
SETB TR0
L1: JNB TF0, L1
CLR P1.5
CLR TR0
CLR TF0
STOP: SJMP STOP
END
```

C Program

```
#include <reg51.h>
int main()
{
TMOD=0X01;
TH0=0X4B;
```

```
TL0=0XFD;

P1.5=1;

TR0=1;

while(TF0==0);

P1.5=0;

TR0=0;

TF0=0;

}
```

Steps to program timer in mode 2 for generating a time delay

1. Load the TMOD register with a value 02h, 20h, 22h indicating which timer (timer 0 or timer 1) is to be used.

2. Load register THx with initial count value

3. Start the timer by setting the TRxbit (i.e) TRx=1

4. Keep monitoring the timer flag (TF) till it is set, with the **JNB TFx,target**instruction to see if it is set.

5. Clear the TF flag for the next round

6.Go back to Step 4 again if delay generation has to be repeated.

3.Write an ALP to generate a square wave of frequency of 4Khz on p2.7 using timer1 in mode2. Assume a XTAL frequency as 11.0592Mhz.

Given frequency of square wave = 4Khz

Required time period of the square wave = 1/4Khz = 0.25ms = 250μs

Required delay = (250μs/2) = 125 μs

N= 125 μs/1.085μs = 115

Initial count = 256-115 = 141

Initial Count in hex = 8Dh

Since Timer1 need to be used in mode2, Value to be loaded in TMOD Register = 20h

```
ORG 00H

MOV TMOD, #20H
```

```
MOV TH1, #8DH
SETB P2.7
SETB TR1
L1: JNB TF1, L1
C PL 2.7
CLR TF1
SJMP L1
END
```

C Program

```
#include <reg51.h>
int main()
{
TMOD=0X20;
TH1=0X8D;
P2.7=1;
TR1=1;
While (1)
{
while(TF1==0);
P2.7=~P2.7;
TF1=0;
}
}
```

3.Write an ALP to generate a square wave of frequency of 100Hz on p2.3 using timer0 of 8051. Assume a XTAL frequency as 11.0592Mhz.

Given frequency of square wave = 100Hz

Required time period of the square wave = 1/100Hz = 0.01 = 10ms

Required delay = (10ms/2) = 5ms

N= 5ms/1.085μs = 4608

Initial count = 65536 – 4608 = 60928

Initial Count in hex = EE00h; TH0 = EEh; TL0 = 00h

Since Timer0 need to be used in mode1, Value to be loaded to TMOD Register = 01h

```
        ORG 00H
        MOV TMOD, #01H
Again: MOV TH0, #EEH
        MOV TL0, #00H
        SETB P2.3
        SETB TR0
L1:     JNB TF0, L1
        CPL P2.3
        CLR TR0
        CLR TF0
        SJMP again
        END
```

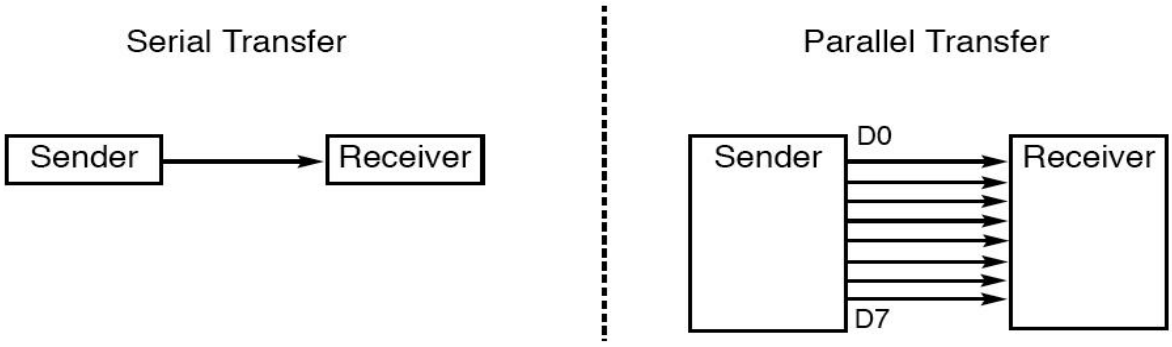
C Program

```
#include <reg51.h>
int main()
{
TMOD=0X02;
P2.3=1;
While (1)
{
```

```
TH0=0XEE;

TL0=0X00;

TR0=1;

While (TF0==0);

P2.7=~P2.7;

TF0=0;

}

}
```

BASICS OF SERIAL COMMUNICATION

Serial communication uses single data line making it much cheaper and enables two computers in different cities to communicate over the telephone. For serial communication to work a byte of data must be converted to serial bits using a parallel-in-serial-out shift register and transmitted over a single data line. At the receiving end there must be a serial-in-parallel-out shift register to receive the serial data and pack them into a byte.



Serial Communication Vs Parallel communication

Paralell Communication	Serial communication
Data transfer of 8 bit width requires 8 data lines	Datatransfer of 8 bit requires only two lines Txd & Rxd
It can be used for short distance communication like printer	Can be used for long distance communication
Requires more hardware making it expensive if the communication distance is more	Cheaper for long distance communication
Faster method of data transfer	Slower method of data transfer
Applications: Communication with printers &harddisks	Applications: Communication with ADC, DAC, EEPROMS etc.

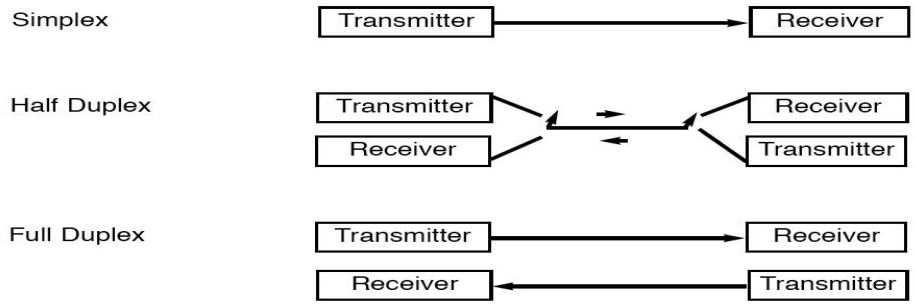
Different modes of Data transfer

Data can be transferred from one device to another in three modes namely Simplex, Half duplex and full duplex.

Simplex: In this mode of data transfer, one device can always transmit and other device can always receive like the communication between computer and printer in which computer can always send the data and printer can always receive data.

Half duplex: In this mode of data transfer, both the devices can transmit and receive but not at the same time.

Full duplex: In this mode of data transfer, both the devices can transmit and receive simultaneously. This mode of data transfer requires two wire conductors for the data lines, one to transmit data and one to receive data simultaneously.



Synchronous & Asynchronous mode of communication

Serial communication uses two methods to transfer data

1. Synchronous communication

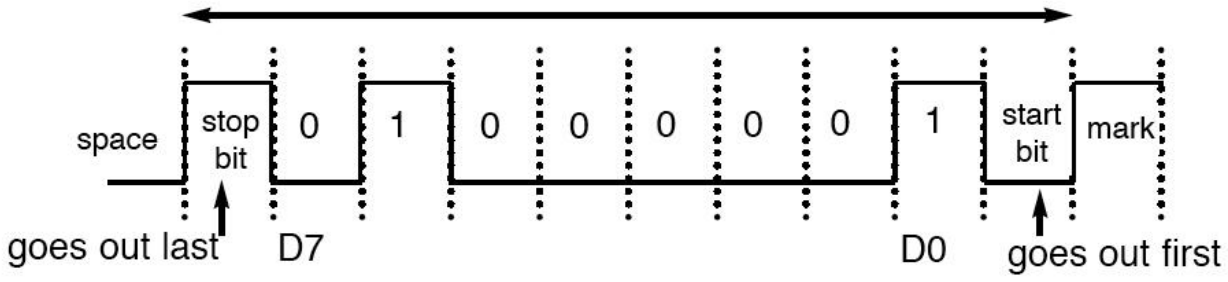
2. Asynchronous communication

Synchronous communication trane of communication transfers a block of data at a time while the asynchronous mode transfers a single byte at a time.

Synchronous serial communication	**Asynchronous serial communication**
Transmitter & Receiver is synchronized by a common clock	Not a common clock is used between transmitter & receiver
Data transmission is faster since the data is transmitted with clock rate	Data transmission is slower since bits are transmitted at constant rate.
Block oriented data transfer	Character oriented data transfer
Sync character is transmitted first and then the data is transmitted	Data is transmitted between start and stop bit as a frame

Asynchronous serial communication and data framing

In Asynchronous method of data transfer, each character is placed between start and stop bits called framing. In data framing the data such as ASCII characters are packed in between a start bit and a stop bit. Start bit is always one bit and stop bit can be one or two bits. Start bit is generally a low signal(0) and stop bit(s) is generally logic high signal(1).

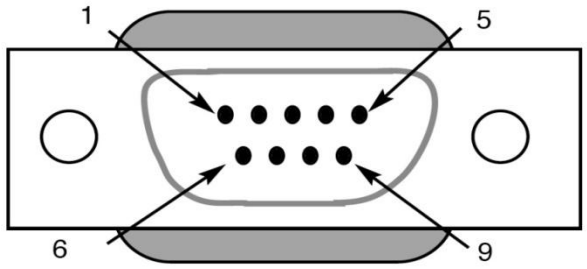


In Asynchronous data transfer start bit is transmitted first and then followed by the data bits in which LSB (D0) is sent out first followed by other consecutive data bits of a byte through the MSB (D7) and lastly the stop bit(s) which indicates the end of character transmission. In this mode of data transfer data bits are transmitted at constant bit rate called baud rate. Baud rate is a way of specifying the rate at which data bits are shifted in and out of a serial device. This method of communication uses a set of rules called protocol for transfer of data which defines the information like how the data is packed, how many bits per character and when the data starts and ends. EIA Introduced a standard for Asynchronous serial communication called RS232 standard.

RS232 Standards

It is the most widely used serial I/O interfacing standard introduced by Electronics Industries Association(EIA) to transfer data between two devices DTE(generally Personal Computer) and DCE (MODEM) with a distance limit of 50 to 100 feet. In this standard, logic 1 bit is represented by -3 to -25 V, logic 0 bit is represented by +3 to +25 V and -3 to +3 is undefined. Since input and output voltage levels of RS232 standard are not TTL compatible, to connect a microcontroller system using RS232 cable must use voltage converters such as MAX232 to convert the TTL logic levels to the RS232 voltage levels, and vice versa. MAX232 IC chips are commonly referred to as line drivers. Generally RS232 cables are generally available in plug connectors namely DB25 and DB-9. DB25 connector uses 25 pins in which most of the pins are unused and DB9 connector uses 9 pins only as shown in figure.

Pin	Description
1	Data carrier detect ($\overline{DCD}$)
2	Received data (RxD)
3	Transmitted data (TxD)
4	Data terminal ready (DTR)
5	Signal ground (GND)
6	Data set ready ($\overline{DSR}$)
7	Request to send ($\overline{RTS}$)
8	Clear to send ($\overline{CTS}$)
9	Ring indicator (RI)



Function of RS232 signals

RS232 DB – 9 connector uses nine signals to communicate between two devices DTE (Data Terminal Equipment) and DCE (Data Communication Equipment). Each signal in RS232 functions as defined below

1. Data carrier detect (DCD)

Data Carrier Detect is used by a modem to indicate that it has a made a connection with other remote modem, (or has detected a carrier signal)

2. Received data (RXD)

This pin carries incoming data from the serial device (DCE) to the computer (DTE)

3. Transmitted data (TXD)

This pin carries outgoing data from the computer (DTE) to the serial device (DCE)

4 Data terminal ready (DTR)

After the power is turned ON, DTE asserts the Data terminal ready (DTR) signal to inform the DCE (modem) that it is ready

5. Signal ground (GND) Signal ground reference

6. Data set ready (DSR)

When modem is ready, it asserts an active low Data set ready (DSR) signal to the DTE in response to the DTR sent by DTE.

7. Request to send (RTS)

Once DSR signal has been received by DTE from DCE, it makes a request to use the data channel by asserting RTS signal to start the data transmission.

8. Clear to send (CTS)

This pin is used by the serial device (DCE) to acknowledge the computer's (DTE) RTS Signal to indicate that it is also ready for data reception.

9. Ring indicator (RI)

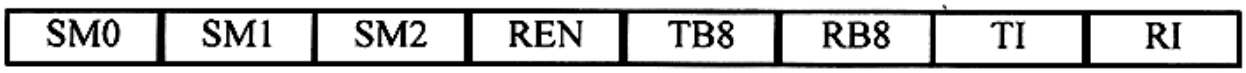
Incoming signal from a modem. It is used by modem (in auto answer mode) to indicate the receipt of atelephone ring signal

After the power is turned ON, DTE asserts the Data terminal ready (DTR) signal to inform the DCE (modem) that it is ready. When modem is ready, it asserts the Data set ready (DSR) signal to the DTE. Once DSR signal has been received, the DTE makes request to use the data channel by asserting RTS signal to start transmission. Then the modem at the other end (receiver end) is dialed. This modem (usually in answer mode) replies by sending a signal at a carrier frequency of 2255 Hz. When the modem at the sending terminal receives this signal, it sends Data carrier detect (DCD) to the DTE, then after the modem sends Clear to send (CTS) showing that the channel is ready for transmission. Immediately after receiving CTS signal, the DTE of the transmitter side sends serial data on its TXD output. For reception of data similar handshaking process is carried out.

Serial communication in 8051 microcontroller

The 8051 has an inbuilt UART to accomplish serial communication. It has two pins for transferring and receiving data by serial communication. These two pins are part of the Port3(P3.0 &P3.1) .These pins are TTL compatible and hence they require a line driver to make them RS232 compatible .Max232 chip is one such line driver in use. UART of 8051 has an 8 bit data register SBUF which is used to hold the data before transmitting the data serially and after receiving the data serially. It has an another 8-bit register called SCON that controls the serial communication of 8051.

SCON register (Bit addressable):



SM0	SCON.7	Serial port mode selector
SM1	SCON.6	Serial port mode selector
SM2	SCON.5	Used for multiprocessor mode communication (not applicable for 8051)
REN	SCON.4	Receive enable. Set or cleared by making this bit either 1 or 0 to enable /disable reception.
TB8	SCON.3	9th data bit transmitted in modes 2 and 3
RB8	SCON.2	9th data bit received in modes 2 and 3.it is not used in mode 0 & mode 1.
TI	SCON.1	Transmit interrupt flag
RI	SCON.0	Receive interrupt flag.

- SM0, SM1 :These two bits of SCON register determine the framing of data by specifying the number of bits per character and start bit and stop bits. There are 4 serial modes.

  SM0 SM1
  0 0 : Serial Mode 0, 8 bit shift register with a fixed baud rate of

XTAL/12

0 1 : Serial Mode 1, 8 bit UART (i.e) 8 bit data,1 stop bit,1 start bit with variable baud rate set by timer1 in mode2

1 0 : Serial Mode 2, 9 bit UART (i.e) 9 bit data,1 stop bit,1 start bit with a fixed baud rate of XTAL/64 or XTAL/32 if baud rate is doubled

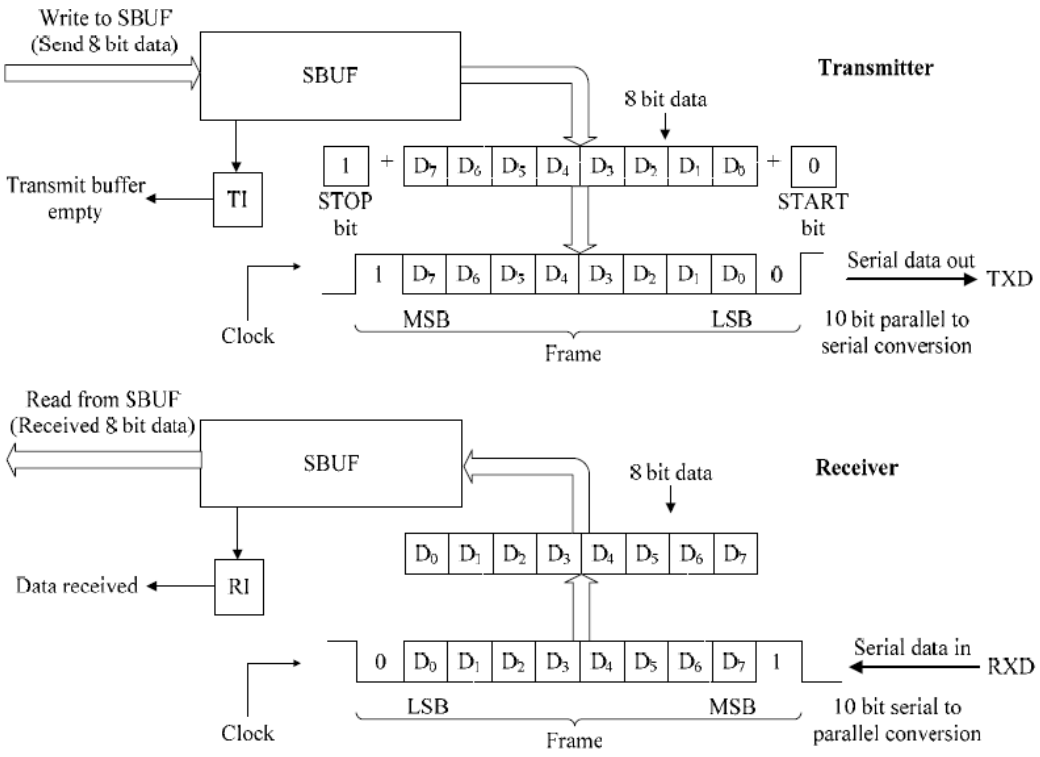
1 1 : Serial Mode 3, 9 bit UART (i.e) 9 bit data,1 stop bit,1 start bit with variable baud rate set by timer1 in mode2.

The baud rate is doubled in mode 1, mode 2 and mode 3 operation of UART if PCON.7 (SMOD) is set.

- REN (Receive Enable) also referred as SCON.4. When it is high, it allows the 8051 to receive data on the RxD pin.
- TI (Transmit interrupt flag) is the D1 bit of SCON register. When 8051 finishes the transfer of 8-bit character, it raises the TI flag to indicate that it is ready to transfer another byte. The TI bit is raised at the beginning of the stop bit.
- RI (Receive interrupt flag) is the D0 bit of the SCON register. When the 8051 receives data serially ,viaRxD, it gets rid of the start and stop bits and places the byte in the SBUF register. Then it raises the RI flag bit to indicate that a byte has been received and should be picked up before it is lost. RI is raised halfway through the stop bit.

Mode 1 operation of UART

UART is designed for mainly for this mode and frame format of this mode is compatible with COM port of PCs. This mode transmits 10 bits through TXD pin and receives through RXD pin as follows: a START bit (always 0), 8 data bits (LSB first) and a STOP bit (always 1). Programmer can set its transmission/reception rate using Timer 1. The process of data transmission and reception is illustrated in figure



Transmission:

Data transmission begins by writing data to the SBUF register. The START and STOP bits are added by hardware to form a 10 bit frame. One bit at a time is transmitted through TXD pin starting from the START bit, then 8 data bits from LSB through MSB followed by STOP bit. After transmitting the complete frame, TI flag is set automatically by serial port hardware to indicate the end of data transmission. We need to monitor TI flag to conform that SBUF register is not overloaded. If TI flag is set, it implies that last character transmission is completed and now SBUF is empty and new byte can be written to it to start next transmission. If a new byte is written to SBUF before TI is raised, the untransmitted part of previous byte will be lost. It should be noted that microcontroller sets TI flag when it completes byte transfer, whereas it must be cleared by the programmer after next byte is loaded in to SBUF.

Programming steps for transmission:

1. Load the register TMOD with 20h to configure timer 1 in mode 2 for baud rate generation.

2. Load the register TH1 with a count N to configure UART with the required baud rate.
3. Start timer 1 by setting the bit TR1 of TCON register.
4. Load the register SCON with 40h or 50h to configure UART in mode 1.
5. Load the data to be transmitted to the register SBUF.
6. Monitor the bit TI of SCON register till it is set to know whether the data in SBUF is transmitted completely via TXD pin.
7.Clear the bit TI.
8. To transmit a new byte of data repeat from step 5.

Reception:

The data reception begins when REN=1 and high to low transition (start bit) is detected on RXD pin. The received byte is loaded into SBUF register by removing the START and STOP bits by UART hardware once complete frame is received. After loading the received byte to SBUF, UART raises the receive Interrupt flag RI to indicate that a valid character byte is available in SBUF. The content of SBUF is copied to any other 8051 register for further processing. RI flag must be cleared immediately and receiver circuit waits for next start bit.

Programming steps for receiving data serially

1. Load the register TMOD with 20h to configure timer 1 in mode 2 for baud rate generation.
2. Load the register TH1 with a count N to configure UART with the required baud rate.
3. Start timer 1 by setting the bit TR1 of TCON register.
4. Load the register SCON with 40h or 50h to configure UART in mode 1.
5.Monitor the bit RI of SCON register till it is set to know whether SBUF is having the received data
6.Clear the bit RI.
7. Load the data from SBUF to accumulator.
8. To receive a new byte of data repeat from step 5.

Calculating the count N to be loaded to TH1 for baud rate generation in mode1

1. Divide the required baud rate from 28,800. (i.e) N1=28,800/ required baud rate
2. Subtract N1 from 256 to get the count N to be loaded to TH1 for baud rate generation.
   (i.e) N=256-N1.

With XTAL = 11.0592 MHz, find the TH1 value needed to have the following baud rates. (a) 9600 (b) 2400 (c) 1200

Solution:

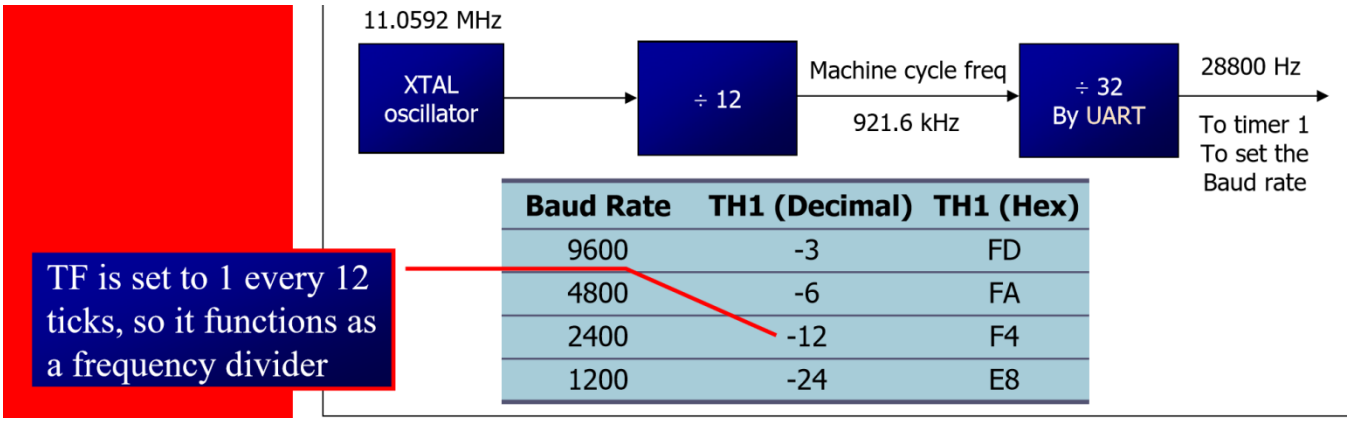
The machine cycle frequency of 8051 = 11.0592 / 12 = 921.6 kHz, and 921.6 kHz / 32 = 28,800 Hz is frequency by UART to timer 1 to set baud rate.

(a) 28,800 / 3 = 9600 where -3 = FD (hex) is loaded into TH1

(b) 28,800 / 12 = 2400 where -12 = F4 (hex) is loaded into TH1

(c) 28,800 / 24 = 1200 where -24 = E8 (hex) is loaded into TH1

Notice that dividing 1/12 of the crystal frequency by 32 is the default value upon activation of the 8051 RESET pin.



Baud Rate	TH1 (Decimal)	TH1 (Hex)
9600	-3	FD
4800	-6	FA
2400	-12	F4
1200	-24	E8

Write a program for the 8051 to transfer letter "A"serially at 4800 baud, continuously.

Solution

```
ORG 00H
MOV  TMOD,#20H  ;timer 1,mode 2(auto reload)
MOV  TH1,#-6    ;4800 baud rate
```

```
        MOV  SCON,#50H  ;8-bit, 1 stop, REN enabled
        SETB TR1        ;start timer 1
AGAIN: MOV  SBUF,#"A";letter "A"to transfer
 HERE:  JNB  TI,HERE    ;wait for the last bit
        CLR  TI         ;clear TI for next char
        SJMP AGAIN      ;keep sending A
        END
```

Write a program for the 8051 to transfer "YES"serially at 9600 baud, 8-bit data, 1 stop bit, do this continuously

Solution:

```
        MOV  TMOD,#20H  ;timer 1,mode 2(auto reload)
        MOV  TH1,#-3    ;9600 baud rate
        MOV  SCON,#50H  ;8-bit, 1 stop, REN enabled
        SETB TR1        ;start timer 1
AGAIN: MOV  A,#"Y";transfer "Y"
        ACALL TRANS
        MOV  A,#"E";transfer "E"
        ACALL TRANS
        MOV  A,#"S";transfer "S"
        ACALL TRANS
        SJMP AGAIN      ;keep doing it
        ;serial data transfer subroutine
TRANS: MOV  SBUF,A      ;load SBUF
        HERE:  JNB  TI,HERE    ;wait for the last bit to be transmitted
        CLR  TI         ;get ready for next byte
        RET
        END
```

Write a program for the 8051 to transfer "GOOD LUCK"serially at 19200 baud, 8-bit data, 1 stop bit, do this continuously

Solution:

```
        MOV  TMOD,#20H  ;timer 1,mode 2(auto reload)
```

```
        MOV  TH1,#-3    ;9600 baud rate
        MOV  SCON,#50H  ;8-bit, 1 stop, REN enabled
        MOV A, PCON
        SETB ACC.7
        MOV PCON, A    ;Double the baudrate by setting PCON.7
        SETB TR1        ;start timer 1
        MOV DPTR, #MES
AGAIN: CLR A
        MOVC A, @A+DPTR
        JZ STOP
        ACALL TRANS  ; Transmit the first character
        INC DPTR
        SJMP AGAIN      ;keep doing it
        ;serial data transfer subroutine
TRANS: MOV  SBUF,A     ;load SBUF
        HERE:  JNB  TI,HERE    ;wait for the last bit to be transmitted
        CLR  TI         ;get ready for next byte
        RET
MES: DB "GOOD LUCK",0
        END
```

Write a program for the 8051 to receive bytes of data serially, and put them in P1, set the baud rate at 4800, 8-bit data, and 1 stop bit

Solution:

```
        ORG 00H
        MOV  TMOD,#20H  ;timer 1,mode 2(auto reload)
        MOV  TH1,#-6    ;4800 baud rate
        MOV  SCON,#50H  ;8-bit, 1 stop, REN enabled
        SETB TR1        ;start timer 1
HERE:  JNB  RI,HERE    ;wait for char to come in
        MOV  A,SBUF     ;saving incoming byte in A
        MOV  P1,A       ;send to port 1
        CLR  RI         ;get ready to receive next   ;byte
        SJMP HERE       ;keep getting data
```

END

Write a C program for 8051 to transfer the letter "A"serially at 4800 baud continuously. Use 8-bit data and 1 stop bit.

Solution:

```
#include <reg51.h>
void main(void)
{
TMOD=0x20;//use Timer 1, mode 2
TH1=0xFA;//4800 baud rate
SCON=0x50;
TR1=1;
while (1)
{
SBUF='A';//place value in buffer
while (TI==0);
TI=0;
} }
```

Write an 8051 C program to transfer the message "YES"serially at 9600 baud, 8-bit data, 1 stop bit. Do this continuously.

Solution:

```
#include <reg51.h>
voidSerTx(unsigned char);
void main(void)
{
TMOD=0x20;//use Timer 1, mode 2
TH1=0xFD;//9600 baud rate
SCON=0x50;
TR1=1;//start timer
while (1)
{
SerTx('Y');
SerTx('E');
```

```
SerTx('S');
} }
voidSerTx(unsigned char x)
{
SBUF=x;    //place value in buffer
while (TI==0);  //wait until transmitted
TI=0;}
```

Write an 8051 C Program to send the two messages "Normal Speed" and "High Speed"to the serial port. Assuming that SW is connected to pin P2.0, monitor its status and set the baud rate as follows:

SW = 0, send the message "Normal Speed" with 28,800 baud rate

SW = 1, send the message "High Speed" with 56K baud rate Assume that XTAL = 11.0592 MHz for both cases.

Solution:

```
#include <reg51.h>
sbit MYSW=P2^0;//input switch
void main(void)
{ unsigned char z;
unsigned char Mess1[]="Normal Speed";
unsigned char Mess2[]="High Speed";
TMOD=0x20; // use Timer 1, mode 2
TH1=0xFF; //28800 for normal
SCON=0x50;
TR1=1; //start timer
if(MYSW==0)
{
for (z=0;z<12;z++)
{
SBUF=Mess1[z]; //place value in buffer
while(TI==0);  //wait for transmit
TI=0; } }
else
```

```
{
PCON=PCON|0x80;//for high speed of 56K
for (z=0;z<10;z++)
{
 SBUF=Mess2[z]; //place value in buffer
while(TI==0);  //wait for transmit
 TI=0; } }
}
```

Program the 8051 in C to receive bytes of data serially and put them in P1. Set the baud rate at 4800, 8-bit data, and 1 stop bit.

Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char mybyte;
TMOD=0x20;//use Timer 1, mode 2
TH1=0xFA;//4800 baud rate
SCON=0x50;
TR1=1;//start timer
while (1)
{            //repeat forever
while (RI==0);//wait to receive
mybyte=SBUF;//save value
 P1=mybyte;//write value to port
RI=0;
 } }
```